
Who Reaches for Combinatorial Testing, and What They Need From It

A user-centered review of `UnitTestDesign.jl`, with experiments and a roadmap

Prepared for	Andrew Dolgert
Prepared by	Claude (Opus 5.5), working in the repository
Date	September 23, 2026
Reviewed	<code>UnitTestDesign.jl</code> v0.4.0, commit <code>e0504d6</code> , branch <code>feature/upgrade</code>
Environment	Julia 1.12.5 on macOS

Statements marked *measured* come from experiments I ran during this review; Appendix A lists them. Function names in the proposal sections (`TestSpace`, `coverage`, `diagnose`, and so on) are sketches of a possible interface, not existing API. I did not read the earlier `interface_*` analyses in the repository.

Contents

Summary	2
1 How I did this review	3
2 The moment someone reaches for combinatorial testing	3
2.1 The situation it fits	3
2.2 The moments that trigger it	4
2.3 Where it does not fit	4
3 Who the users are	5
3.1 Users the documentation already imagines	5
3.2 Users no one has addressed	6
4 What users worry about, and how the package does today	6
5 Evidence	7
5.1 Constraints	7
5.2 GND picks most values at random	9
5.3 Mixed strength works, but its interface is fragile	9
5.4 Performance	9
5.5 What a failure looks like	10
5.6 Reproducibility and stability	10
5.7 Coverage accumulates early, and higher coverage comes free	10
5.8 Error messages	11
6 Communicating what it does, that it did it, and what is possible next	11
6.1 Before: a preview that helps choose a strength	11
6.2 Generate: a result that states its guarantee	12
6.3 Run: cases with names, outcomes that are kept	13
6.4 After: diagnosis, masking, and the next step	13
6.5 For AI agents in particular	14
6.6 Words matter	14
7 A sketch of the interface	14
8 Making the solvers reliable	16
8.1 Two ways generation fails	16
8.2 The fix: feasibility first, then keep every row completable	16
8.3 Rules the solver can understand	16
8.4 Where Z3 fits	17
8.5 Smaller fixes	17
9 Documentation and first contact	17
10 Roadmap	18
11 Closing	18
A Reproducing the experiments	18
B Findings by location	19

Summary

UnitTestDesign.jl answers one question well: *given a list of values for each parameter, which few combinations should I test so that every pair (or triple, or t -tuple) of values appears somewhere?* Its default engine, IPOG, is fast. Fifteen four-valued parameters at strength four take about four seconds and produce 958 tests, within 4% of the 924 reported in the IPOG paper [2]. In each of the 888 constrained random problems where IPOG returned a result, an independent checker confirmed that the result was valid and complete. The core algorithms are sound wherever they finish.

The package's limits lie around generation rather than in it. People reach for combinatorial testing at a particular moment: a nested loop over options has just exploded, or each run costs an hour of cluster time, or a bug arrived from a combination nobody tried. At that moment they have questions. How many tests will this be? Did it really cover everything? What did it leave out, and why? When test 7 fails, which combination is to blame? What would it cost to go further? Today the package returns a `Vector{Vector{Any}}` and says nothing more. Users have to work out everything else themselves.

What I found, in order of consequence:

1. **Constraints are where users get hurt.** With a disallow rule, IPOG crashed with `BoundsError: attempt to access ... at index [0]` on 8.6% of random pairwise problems and 13.8% of three-way problems (*measured*). Every problem that contained an *implicit* constraint crashed. An implicit constraint arises when two innocent rules together make some pair impossible. On the same kind of input GND does not crash; it loops forever. And because unassigned parameters reach the rule as nothing, a parameter whose real values include nothing (a common Julia default) silently loses required coverage.
2. **GND has a one-token bug.** One comparison uses `n_way` where it needs `n_way - 1`. As a result, once as many parameters are set as the strength (two, for pairwise), GND picks every remaining value at random. Fixing it shrinks GND's designs by 17–35%, and GND then beats IPOG when parameters have eight values (*measured*). The documentation describes GND as the engine that finds shorter designs, but the shipped code finds longer ones.
3. **Parameters have positions but no names.** That single decision makes constraints positional, makes forty-parameter configurations unreadable, and makes a failing case print as `Any[1000, :newton, 0.001, true]`.
4. **Several natural users are not served at all.** These include the maintainer who wants to know what their existing hand-written tests cover, the developer debugging a failure, the engineer sizing a CI matrix, the tester of invalid inputs, and the AI agent writing tests on someone's behalf.

The recommendation, in one sentence: **turn the package from a generator that returns a list into a short conversation about a test space.** That conversation runs: plan, generate with a stated guarantee, run with names, diagnose, extend. Each step reports what it did and what is possible next. Table 1 lists the ten changes I would make first.

Table 1: The ten changes I would make first.

	Change	Effort	Who benefits
1	Fix GND's match condition (one token); seed GND by default and record the seed.	Hours	Everyone using GND
2	Make constraints safe: drop and report impossible tuples, keep every row completable, never loop without progress.	Days	Everyone with rules
3	Return a design object that prints its guarantee, exclusions, and next steps, while still iterating like today's vector.	Days	Everyone; reviewers; agents
4	Named parameters; each case a NamedTuple.	Days	Everyone
5	Constraints scoped by argument names, so rules never see nothing.	Days	Everyone with rules
6	coverage(cases, space) to measure any test set, including hand-written ones; topup to fill the gaps.	Days	Maintainers, auditors
7	run_cases and diagnose: name cases in test output, rank suspect combinations, propose follow-ups.	1–2 weeks	Anyone with a failure
8	Error messages in the user's vocabulary (parameter names and values), checked before generation.	Days	Everyone; especially agents
9	Export to Tables.jl, CSV/TOML, a GitHub Actions matrix, and literal test code.	Days each	CI, simulation, agents
10	Organize docs by job, run examples as doctests, correct the engine guidance, add a one-page agent guide.	Days	First-time users

1 How I did this review

I read all of the source, tests, and documentation, plus the JuliaCon paper. Then I used the package the way a newcomer would, in a scratch environment, and ran experiments whose results appear throughout this report:

- An **independent coverage checker** enumerates every feasible t -tuple by brute force and shares no code with the package. A tuple is *feasible* if at least one complete, allowed test contains it.
- **1,000 random constrained problems** (500 at strength 2 and 500 at strength 3). Each had 4–6 parameters with 2–4 values, and 1–3 forbidden partial assignments over 2–3 parameters. A fixed seed makes the run repeatable.
- **Other checks:** timing and allocation across problem sizes, GND under a watchdog, verification of mixed strength, stability under small edits, and the printed output of failing tests.
- A **throwaway prototype** of about 50 lines, to check that the constraint fix I recommend in Section 8 actually works.

The scripts are small enough to become regression tests.

2 The moment someone reaches for combinatorial testing

2.1 The situation it fits

Combinatorial testing fits when three conditions hold at once.

1. **The inputs are, or can be made into, discrete choices:** options, modes, flags, algorithms, element types, container types, platforms, or representative values of equivalence classes.

2. **Failures come from interactions among a few choices.** Kuhn, Wallace, and Gallo found that the failures they studied were triggered by at most six interacting parameters, and most by one or two [1]. Covering every t -way combination therefore comes close to exhaustive testing at a small fraction of the cost.
3. **Exhaustive testing is unaffordable.** Either the product is huge (twenty four-valued options give about 10^{12} combinations), or each test is expensive (a simulation that runs for an hour).

2.2 The moments that trigger it

Nobody sets out to do combinatorial testing. People arrive at it from a few recognizable situations, and the package's first page should name them.

The loop that exploded.

Someone writes for T in types, A in arrays, alg in algorithms, ..., and the suite goes from seconds to an hour. Or they quietly comment out half the loop.

The expensive run.

Each case is a cluster job. The budget is thirty runs, not three thousand.

The bug from nowhere.

A user reports a crash with a combination of options nobody tried, and the maintainer wonders which other combinations have never run.

The matrix bill.

The CI matrix of operating systems, Julia versions, architectures, thread counts, and dependency versions costs too much. Or it hits GitHub Actions' limit of 256 jobs per matrix [9].

The audit.

A reviewer, funder, or regulator asks how thoroughly the options have been exercised.

Generation serves the first four situations. The audit needs *measurement*, which the package does not yet offer. And every situation eventually leads to a failure that needs *diagnosis*, which it also doesn't offer.

2.3 Where it does not fit

Part of serving users is telling them when another tool is better. The documentation says little about this today.

- **Finding the one value that breaks a numerical routine** calls for property-based testing or fuzzing, for example Supposition.jl [8]. It explores values within domains and shrinks failures to minimal examples.
- **Estimating how much each factor affects an outcome** is design of experiments. Orthogonal arrays and fractional factorials are balanced for estimation, while covering arrays are not. A scientist planning a sensitivity study should be pointed elsewhere.
- **Sweeps over continuous parameters** call for space-filling samples such as Sobol sequences or Latin hypercubes.

Property-based testing is a partner, not a rival. Combinatorial testing decides *which partitions to combine*; property-based testing decides *which values to draw within each partition*. The package's Testing Methods page already hints at this, and it could become a first-class feature (Section 7).

3 Who the users are

Table 2 lists eight kinds of users. The first four are implied by the current documentation. The last four are not addressed at all, though each has a well-established counterpart in the testing literature or in other tools.

Table 2: Who reaches for this package, and whether it serves them today.

User	What they need most	Served today
Generic-code author	Combinations of types and options; readable failures; fast generation inside <code>runtests.jl</code>	Partly
Simulation developer	Few expensive runs; constraints; reference runs included; cases as files for a scheduler	Partly
CI / release engineer	Matrix output; rules such as “no arm64 on Windows”; the same jobs from commit to commit	Not yet
Test-data builder	Tables with named columns that cover value combinations	Partly
Maintainer / auditor	Measure the interaction coverage of existing tests; add only what is missing	Not yet
Debugger	Which combination caused this failure, and what to run next	Not yet
Robustness tester	Invalid values tested one at a time, so one does not mask another	Not yet
AI coding agent	Knowing when to use it; results that describe themselves; checkable claims; actionable errors	Not yet

3.1 Users the documentation already imagines

The generic-code author. This user is especially common in Julia. Multiple dispatch means that combinations of element type, container type, dimension, and option really are different compiled methods. The extended Hilbert-curve example in the documentation is this user’s story, and combinatorial testing found a real bug there. They run generation at test time, so it must be quick and deterministic. They need failures they can read. They care about stability, so that a failure on Tuesday can be compared with Wednesday’s. Positional, unnamed parameters and `Vector{Any}` results work against all of these.

The simulation developer. This is the user in the paper’s statement of need. They have a configuration file with dozens of options, and each run takes minutes to hours, often on a cluster. Their “tests” are rarely `@test` calls. They are jobs, so they need the cases as data (TOML, CSV, JSON) to hand to a scheduler. Their options come with many rules, since some settings only make sense together, and that is exactly where the package is most fragile today. They have reference runs that must be included. They want stronger coverage on risky subsystems, which the `wayness` option provides. When a campaign is half done and some runs crashed, they need to know what remains untested, and the package cannot tell them. Forty positional parameters are also hard to maintain.

The CI and release engineer. Their space is operating system $\times$ Julia version $\times$ architecture $\times$ threads $\times$ dependency versions. They need the design as a GitHub Actions include: list and rules that express what is impossible. They also need stability: if adding a Julia version reshuffles every job, yesterday’s failure can no longer be compared with today’s. There is also an opportunity. A randomized engine seeded with the run number would give a different pairwise matrix on each CI run, so higher-order coverage accumulates across runs at no extra cost.

The test-data builder. The documentation’s DataFrame example builds rows that cover value combinations for testing a data pipeline. Today the user has to convert positional vectors into named columns by hand.

3.2 Users no one has addressed

The maintainer or auditor of an existing suite. Most projects already have tests. The first question a thoughtful maintainer asks is not “generate new tests” but “what do my current tests cover, and what is the smallest set I could add?” NIST built a tool, CCM, for exactly this question [4]. UnitTestDesign.jl already has the internal machinery: coverage structures, and seeds that count toward coverage. It exposes none of it. A coverage function and a topup function would serve this user without disturbing how they already test, and this is the gentlest way to bring combinatorial testing into a project.

The debugger. After a failure, the natural question is *which combination caused it?* A covering design is unusually good at answering this. Candidate causes are combinations that appear in failing cases but never in passing ones, and a few follow-up tests can separate them. Tools such as BEN build on this idea [5]. There is a second point, known as *masking*. Tuples that appeared only in failing cases have not been shown to work, so the coverage guarantee is weaker than it looks until they are re-tested elsewhere [6]. Section 5.5 shows what such a report could say for a real run.

The robustness tester. Invalid inputs (negative sizes, empty strings, NaN tolerances) should appear *one per test case*. Otherwise the first invalid value triggers an error and hides the others. Microsoft’s PICT supports this by letting users mark values as invalid [7]. It is a small addition that changes what the package is good for.

The AI coding agent. Agents now write a large share of tests. Left alone, they tend either to write the Cartesian loop that explodes or to hand-pick three cases, and this package is the principled middle path. An agent needs the package to (i) state in its docstrings the situation it is for, so the agent knows to reach for it; (ii) return results that describe themselves in text, so the agent can report accurately; (iii) be deterministic, so reruns match; (iv) produce errors that name the parameter and suggest a fix, because agents act on messages literally; (v) offer a `verify` or coverage function, so claims can be checked rather than trusted; and (vi) optionally emit the design as literal test code, so a human reviewer sees every case in the diff. The human who reviews an agent’s tests needs (ii) and (v) just as much.

4 What users worry about, and how the package does today

Whatever brings them to the package, users share a common set of concerns. Table 3 lists each one with an assessment and a pointer to the evidence.

Table 3: Users’ concerns, rated against version 0.4.0.

The user’s question	Today	Evidence
Is everything really covered?	Partly	Whenever a design is returned, the guarantee holds: 888 of 888 constrained designs, and mixed strength (§5.3). But nothing lets the user see or check that; no coverage function is exported.
How many tests will this be, and is that good?	Not yet	There is no preview, no comparison with the full factorial, and no lower bound. <code>full_factorial</code> allocates before checking size: sixteen four-valued parameters would need 550 GB.

The user's question	Today	Evidence
Will it respect what my function accepts?	Not yet	IPOG crashes on 8.6% and 13.8% of constrained problems; GND hangs; natural rule code throws; a real nothing value silently loses coverage (§5.1).
Will my must-run cases be included?	Partly	Yes. But a typo in a value produces <code>MethodError: Cannot convert ... Nothing to ... Int64</code> ; must-run cases that break the rules are kept silently; a forbidden excursion base case silently disappears.
Will I get the same tests tomorrow?	Partly	IPOG is deterministic. GND is unseeded by default, so two calls differ. Adding one value to one parameter kept only 3 of 10 IPOG cases (§5.6).
When a case fails, which one was it?	Not yet	The README loop reports “2 errors” with no parameters. The per-case pattern prints <code>Any[1000, :newton, 0.001, true]</code> (§5.5).
Which combination caused it?	Not yet	Nothing is provided, although the data to answer it is already in hand.
Can I fit this in my budget?	Partly	IPOG puts most coverage early (16 of 28 cases reach 90% of pairs), but this isn't documented, and there is no <code>max_cases</code> (§5.7).
Can I test the risky parts harder?	Partly	Mixed strength works and is verified. But the documented example throws, the user's <code>Dict</code> is modified, and GND fails an assertion in one case (§5.3).
Can I test invalid inputs properly?	Not yet	There is no notion of invalid values.
Which engine should I use?	Not yet	The documentation says GND is shorter. It is 26–38% longer, because of a bug (§5.2).
Is it light and fast?	Met	Two dependencies. IPOG runs in well under a second for typical sizes (§5.4).
Will it fit my tools?	Partly	It works inside a <code>for</code> loop. It has no names, no <code>Tables.jl</code> support, no file or matrix export, and no <code>Test.jl</code> integration.

5 Evidence

5.1 Constraints

The nothing convention

Rules are ordinary Julia functions over all parameters. During generation, the package calls a rule on partial test cases and passes nothing for parameters it has not chosen yet (`src/factorial_interface.jl:85`). The guide warns about this, but natural rule code still breaks:

```
all_pairs([1,2,3], [0.0, 1.0, 10.0], [:a,:b];
          disallow = (n, x, s) -> n > 2 && x > 1.0)
# MethodError: no method matching isless(::Float64, ::Nothing)
```

The convention has a subtler cost as well. In Julia, `nothing` is a common *real* value, as in `maxiter = nothing`. When a parameter takes `nothing` as one of its values, the package cannot tell “not yet chosen” from “chosen to be nothing”. It then drops required tuples without saying so:

```
maxiter = [nothing, 10, 100]; mode = [:fast, :slow]
verbose = [true, false]; scale = [1.0, 2.0, 3.0]
dis = (m, md, v, s) -> m === nothing && md === :slow
```

```
cases = all_pairs(maxiter, mode, verbose, scale; disallow = dis)
# Independent check: 35 of 36 feasible pairs covered.
# Never tested: mode = :slow, scale = 1.0 (a perfectly valid pair)
```

Random constrained problems

Table 4 summarizes 1,000 random problems. IPOG never returned a wrong answer. It either returned a correct one or crashed. For every crash I checked whether some tuple that the rule permits in isolation is impossible in any complete, allowed test, which is the definition of an implicit constraint. The rates depend on how tightly the rules bind, and these small, fairly constrained problems are not a sample of real projects. The failure classes are real and easy to hit, as the next example shows.

Table 4: IPOG on 1,000 random constrained problems (*measured*).

	Strength 2	Strength 3
Problems	500	500
Returned; independently verified valid and complete	457	431
Returned but invalid or incomplete	0	0
Crashed with BoundsError	43 (8.6%)	69 (13.8%)
of which contained an implicit constraint	28	65
of which were greedy dead ends (no implicit constraint)	15	4
Problems with an implicit constraint that did <i>not</i> crash	0	0
Prototype with completion checks (§8): valid and complete	500	500

Implicit constraints are ordinary in practice. Here is a three-parameter example:

```
os = [:linux, :mac, :windows]; gpu = [false, true]; driver = [:cuda, :none]
dis(o, g, d) = (g === true && d === :none) || (o === :windows && d === :cuda)
all_pairs(os, gpu, driver; disallow = dis)
# BoundsError: attempt to access 2-element Vector{Symbol} at index [0]
all_pairs(os, gpu, driver; disallow = dis, engine = GND())
# never returns (killed by a 45-second watchdog; 12 combinations in total)
```

Neither rule mentions Windows together with a GPU. But a GPU needs CUDA and Windows forbids CUDA, so no valid test can pair `os = :windows` with `gpu = true`. The user sees an index error that points into the package’s internals, or a process that never finishes.

Why it happens. IPOG keeps any tuple that the rule does not reject on its own. When the tuple fits nowhere, IPOG appends it as a new row (`src/parameter_order.jl:170`). The final fill then finds no allowed value for some slot and leaves a zero (lines 224–226). Converting back to values then indexes `p[0]`. GND’s outer loop (`src/greedy_tuples.jl:195`) waits forever for a tuple no candidate can cover. Its inner loop (line 201) can also spin if every candidate is forbidden. The greedy dead ends have a related cause: values are placed after checking only that the row is not *yet* forbidden, not that it can still be completed. The test suite exercises only simple rules over two parameters, so neither failure is caught.

Silent acceptance

Two more behaviors pass without a word. Must-run cases (seeds) that violate the rules are included in the output anyway. And when the base case of an excursion is forbidden, it is dropped, even though the base case is the whole point of an excursion.

5.2 GND picks most values at random

`most_matches_existing` (`src/coverage_matrix.jl:193`) scores each candidate value by how many uncovered tuples it would complete. The comment above it says a tuple can match at most $n - 1$ known values, because the parameter being chosen is still unset. The code, however, tests `match_cnt == min(max_known, n_way)`. Once the number of set parameters reaches the strength (two, for pairwise), no tuple satisfies this, every score is zero, and the tie-breaker picks a random value. GND then relies on its M random restarts to find anything good.

Table 5: Number of test cases: IPOG, GND as shipped, and GND with $n_way - 1$ (three seeds each; *measured*).

Parameters $\times$ values, strength	IPOG	GND as shipped	GND patched
$10 \times 4, t = 2$	28	36–38	30
$10 \times 8, t = 2$	113	151–153	100
$20 \times 4, t = 2$	37	48–51	37
$10 \times 4, t = 3$	143	180–186	144–150

Increasing M does not close the gap. On the 10×4 problem, GND as shipped gave 40–41 cases at $M=10$, 36–38 at 50, 32–34 at 200, and 32–33 at 1000 (1.2 s). Even twenty times the default effort did not reach IPOG’s 28. The patched version still covered all 720 pairs when checked independently.

5.3 Mixed strength works, but its interface is fragile

`all_pairs(fill(1:4, 10)...; wayness = Dict{3 => [[3,4,5,6]]})` produced 64 cases (IPOG) and 84 (GND). The independent check confirmed all 720 pairs and all 256 triples on parameters 3–6. The feature itself is sound. Around it:

- The guide’s own example, `Dict{3 => [[3:6], [25:30]]}`, throws `MethodError: Cannot convert an object of type Int64 to an object of type UnitRange{Int64}`.
- The user’s dictionary is modified. After a call it has gained an entry `2 => [[1, 2, 3, 4]]` (`src/parameter_order.jl:436`, `src/excursions.jl:32`), so reusing it in a later call changes that call’s meaning.
- GND fails with `AssertionError: minimum(orders) > base_wayness` if any listed strength equals the base strength (`src/coverage_matrix.jl:426`).

5.4 Performance

IPOG is fast enough that generation can run inside a test suite (Table 6). Allocation is heavy because hot loops slice matrix columns, but it isn’t a practical problem at these sizes. The one real risk is `full_factorial`, which allocates its whole result before checking whether the size is reasonable.

Table 6: Generation time and total allocation (*measured*; times include some compilation; allocation is cumulative, not peak).

Problem	IPOG cases	IPOG time	Allocated	GND cases	GND time
$10 \times 4, t = 2$	28	0.12 s	42 MB	36–38	0.08 s
$20 \times 4, t = 2$	37	0.11 s	42 MB	48–51	0.21 s
$50 \times 5, t = 2$	75	0.24 s	0.4 GB	–	–
$100 \times 3, t = 2$	33	0.37 s	0.5 GB	–	–
$10 \times 4, t = 3$	143	0.11 s	87 MB	180–186	1.02 s
$30 \times 4, t = 3$	256	1.55 s	5.7 GB	–	–
$50 \times 3, t = 3$	136	1.87 s	8.6 GB	–	–
$15 \times 4, t = 4$	958	4.06 s	17 GB	–	–

5.5 What a failure looks like

Here is a solver with a two-way bug. It fails only when `method = :newton` and `sparse = true`.

```
solve(n, method, tol, sparse) =
  (method == :newton && sparse) ? error("singular factorization") : n * tol
cases = all_pairs([10, 100, 1000], [:newton, :big, :gmres], [1e-3, 1e-6], [false, true])
```

With the README’s loop, Test.jl reports `solver | Pass 8 Error 2` and two stack traces for the expression `solve(tc...) > 0`. The output never says which parameters failed. With the documentation’s per-case pattern (`@testset "solver $tc" for tc in cases`), the failing case prints as `solver Any[1000, :newton, 0.001, true]`, which a reader has to decode by position.

The design and the outcomes already contain the answer. About twenty lines of code, run on this very output, find the combinations that appear in failing cases but never in passing ones:

```
suspicious: method=:newton, sparse=true      (in 2 of 2 failures, 0 of 8 passes)
suspicious: n=1000, method=:newton           (in 1 of 2 failures, 0 of 8 passes)
suspicious: n=100, sparse=true                (in 1 of 2 failures, 0 of 8 passes)
```

The true cause ranks first. The other two pairs are *masked*: they appeared only in failing cases, so nobody knows whether they work. Two follow-up cases would settle all three: `(1000, :newton, 0.001, false)` and `(100, :gmres, 1e-6, true)`. Section 6 turns this into a proposed feature.

5.6 Reproducibility and stability

GND() seeds itself from the global MersenneTwister() (`src/factorial_interface.jl:61`). Two calls with identical arguments gave different designs, so a failure found with GND cannot be reproduced unless the user thought to pass an RNG in advance. IPOG is deterministic but not stable: adding one value to one of five parameters changed the design from 10 to 14 cases, and only 3 of the original rows survived. This matters to anyone who compares failures across commits. Passing the old design as seeds would preserve it, but nothing suggests doing so.

5.7 Coverage accumulates early, and higher coverage comes free

The order of the cases matters for anyone on a budget. On 10×4 pairwise, IPOG’s first cases cover pairs quickly:

First j cases	4	8	12	16	20	24	all
IPOG, share of pairs covered	25%	47%	68%	90%	93%	97%	100% (28)
GND as shipped	25%	47%	64%	77%	86%	92%	100% (37)

A pairwise design also covers a good share of triples as a side effect: 25% for 5×4 , 40% for 10×4 , 54% for 10×3 , and 61% for 20×3 (*measured*). Users never see either number, yet both answer the questions “can I stop early?” and “how much am I getting?”

5.8 Error messages

Most error messages come from deep inside the algorithms and use the algorithms’ vocabulary, not the user’s (Table 7).

Table 7: What users see, and what would help (*measured*).

Input	What the user sees	What would help
A parameter with one value, [true]	DomainError with Bool[1]: Each argument should be a list of parameter values	Accept it; a fixed setting is legitimate.
Strength 3 with two parameters	BoundsError: attempt to access 2-element Vector{Int64} at index [1:3]	“Strength 3 needs at least 3 parameters; you gave 2.”
Must-run case with "medium", which is not a listed value	MethodError: Cannot ‘convert’ an object of type Nothing to an object of type Int64	“must_include case 1: "medium" is not a value of level (values: "low", "mid", "high").”
Rule (n, x, s) -> n > 2 && x > 1.0	MethodError: no method matching isless(::Float64, ::Nothing)	Unreachable with scoped rules (§8).
Implicit constraint	BoundsError: attempt to access 2-element Vector{Symbol} at index [0]	“os = :windows, gpu = true cannot appear in any valid case (rules 1 and 2 together); left out.”
Same, with GND	Never returns	The same message.
The guide’s wayness example	MethodError: Cannot ‘convert’ an object of type Int64 to an object of type UnitRange{Int64}	Accept ranges.
wayness at the base strength, GND	AssertionError: minimum(orders) > base_wayness	Ignore the redundant entry, or explain it.

6 Communicating what it does, that it did it, and what is possible next

This is the center of your question. A user passes through four stages, and at each one they have a question the package can answer better than they can. The design principle is simple: **answer the question at the moment it arises, in the user’s own vocabulary, and point to the next useful step.**

6.1 Before: a preview that helps choose a strength

The decision a user struggles with is the strength t . The package can make it concrete, because IPOG is fast enough to simply try each option. For a CI space of five parameters with two rules, the numbers below are real (*measured*). Only the presentation is a proposal.

Table 8: The user’s question at each stage, and who answers it.

Stage	The user’s question	Answered today by
Before	Is this the right tool? What will it cost? Which strength should I choose?	The user’s own arithmetic
Generate	Did it keep its promise? What did it leave out, and why? Can I reproduce it?	Nothing
Run	Which case is this? Which cases failed, by name?	A positional Vector{Any}
After	What caused the failure? What remains unverified? What would it cost to go further?	Nothing

```
julia> compare_strengths(space; forbid = rules)
5 parameters (2-3 values each): 72 combinations, 48 allowed by the rules.
```

strength	cases	share of allowed	pairs covered	triples covered
1	3	6%	49%	24%
2	10	21%	100%	69%
3	20	42%	100%	100%
4	39	81%	100%	100%

Most failures involve 1-2 parameters (Kuhn et al. 2004); strength 2 is the usual start. Raise strength for risky subsets with `stronger = ...`.

6.2 Generate: a result that states its guarantee

The most valuable single change is to return an object that is still an AbstractVector, so every existing for loop keeps working, but that prints a summary. Here is the same CI example. Every number comes from the real run.

```
julia> design = all_pairs(space; forbid = [
    (os, gpu) -> os == :mac && gpu,
    (os, arch) -> os == :windows && arch == :arm64])
TestDesign: 10 cases over 5 parameters (strength 2, IPOG, deterministic)
Guarantee   every feasible pair of values appears in at least one case
             55 of 55 feasible pairs covered (checked)
Space       72 combinations in full; 48 satisfy the rules
Left out    2 pairs forbidden by rules:
             os = :mac,      gpu = true      rule 1
             os = :windows, arch = :arm64    rule 2
Bonus       83 of 120 feasible triples also covered (69%)
Budget      first 5 cases cover 73% of pairs; first 8 cover 95%
Next        strength 3 needs 20 cases: extend(design; strength = 3)
```

#	os	julia	threads	gpu	arch
1	:linux	"1.10"	1	false	:x64
2	:linux	"1.11"	4	true	:arm64
3	:linux	"1.12"	1	true	:x64
4	:mac	"1.10"	4	false	:arm64
5	:mac	"1.11"	1	false	:x64
... 5 more					

When the rules make a pair impossible, the “Left out” section says so and says why. It distinguishes pairs a rule forbids directly from pairs that are impossible only because rules combine. The prototype already produces this list for the three-parameter driver example:

```
Left out    3 pairs cannot appear in any valid case:
            os = :windows, driver = :cuda      rule 2
            gpu = true, driver = :none         rule 1
            os = :windows, gpu = true         impossible: rules 1 and 2 together
```

This one screen answers “did it keep its promise”, “what did it leave out”, “can I stop early”, and “what next”. The user does not have to ask.

6.3 Run: cases with names, outcomes that are kept

With named parameters, each case is a `NamedTuple`, so test output reads (`n = 1000`, `method = :newton`, `tol = 0.001`, `sparse = true`). A small helper can run the body for every case. It gives each case its own named testset, keeps going after failures, and keeps each outcome next to the design:

```
@testset "solver" begin
    global results = run_cases(design) do c
        @test solve(c.n, c.method, c.tol, c.sparse) > 0
    end
end
```

Simulation developers do not run cases inside `Test.jl`, so the same idea needs a file form. They write the design out, run the jobs, and read the outcomes back in (`record_outcomes(design, "results.csv")`), so that diagnose and coverage work on campaigns as well as unit tests.

6.4 After: diagnosis, masking, and the next step

This output is again computed from the real run in Section 5.5:

```
julia> diagnose(results)
2 of 10 cases failed, both with ErrorException("singular factorization").

Combinations present only in failing cases:
  method = :newton, sparse = true      2 of 2 failures, 0 of 8 passes    <- prime suspect
  n = 1000, method = :newton           1 of 2 failures
  n = 100, sparse = true                1 of 2 failures

Not yet verified: the last two pairs appeared only in failing cases.
Two follow-up cases would settle all three:
  (n = 1000, method = :newton, tol = 0.001, sparse = false)
  (n = 100, method = :gmres, tol = 1.0e-6, sparse = true)
Run them with: run_cases(followups(results)) do c ... end
```

Three more functions complete the loop after a run:

- `extend(design; strength = 3)` keeps the existing cases and adds only what the higher strength needs. The seed mechanism already does this internally.
- `coverage(cases, space; strength = 2)` measures *any* list of cases, including hand-written ones, and lists what is missing by name.

- `topup(cases, space)` returns only the new cases needed to reach full coverage, for maintainers adopting the package gradually.

6.5 For AI agents in particular

Everything above helps agents, and a few further things matter especially to them:

- **A machine-readable summary.** `summary(design)` returns a `NamedTuple` (`cases`, `strength`, `engine`, `seed`, `required`, `covered`, `forbidden`, `impossible`, `full size`) that serializes to JSON, so an agent can report exact figures instead of paraphrasing.
- **Checkable claims.** coverage is independent of the generator, so an agent, or the human reviewing its work, can verify the guarantee in one line.
- **Docstrings that begin with the situation.** For example: “Use when a function or system has several discrete options and testing every combination is too slow.” Agents choose tools by matching situations.
- **Errors that name things and suggest a fix.** Agents repair code from messages, and a message that names `level` and lists its values gets fixed on the first try.
- **Literal output.** `as_code(design)` prints the cases as a Julia literal, or as a `@testset` block, so the committed test file shows every case. Reviewers see exactly what runs, and the test has no hidden generation step.
- **A one-page guide in the repository** (an `llms.txt`-style file) giving the decision rule and three canonical patterns: test a function, plan a CI matrix, audit existing tests.

6.6 Words matter

Some of the current names get in the way of understanding (Table 9). The old names can stay as aliases.

Table 9: Suggested vocabulary.

Today	Problem	Suggested
seeds	Collides with random-number seeds. The package itself uses both, and so does its test suite (<code>seed_mod</code>).	<code>must_include</code>
<code>n_way</code>	Fine for experts; the literature says “strength”.	<code>strength</code> (keep <code>n_way</code>)
<code>wayness = Dict{3 => [[3,4,5,6]]}</code>	Dictionary of lists of lists of positions.	<code>stronger = [(:a, :b, :c) => 3]</code>
<code>disallow</code> over all positions	Must handle nothing; the scope is invisible.	<code>forbid</code> rules over named arguments
“coverage”	Collides with line coverage.	“interaction coverage”
IPOG, GND	Named by algorithm, not by behavior.	Keep, but document as “deterministic” and “randomized, varies by seed”

7 A sketch of the interface

The sketch follows four principles. It stays backward compatible: today’s positional calls keep working. It prefers names over positions. The result is the vector, so loops keep working. And every function returns something that prints a useful summary.

```

using UnitTestDesign

space = TestSpace(
    os      = [:linux, :mac, :windows],
    julia    = ["1.10", "1.11", "1.12"],
    threads  = [1, 4],
    gpu      = [false, true],
    arch     = [:x64, :arm64],
)

design = all_pairs(space;
    forbid      = [(os, gpu) -> os == :mac && gpu,           # scope = argument names
                  (os, arch) -> os == :windows && arch == :arm64],
    must_include = [(os = :linux, julia = "1.12", threads = 4, gpu = true, arch = :x64)],
    stronger     = [(:julia, :threads, :gpu) => 3],
)

design[3]                # (os = :linux, julia = "1.12", threads = 1, ...)
coverage(design)         # checked independently of the generator
coverage(handwritten, space) # audit an existing suite
topup(handwritten, space)  # only the cases needed to complete it
extend(design; strength = 3) # keep these cases, add the rest

github_matrix(design)     # JSON for a workflow's `include:` list
write_design("campaign.toml", design); read_design("campaign.toml")
as_code(design)           # literal cases for a reviewable test file

```

Invalid values. A value wrapped as `Invalid(-1)` would be placed with only valid values around it, and every invalid value would still be paired with every valid value of the other parameters [7]. The summary would report the two guarantees separately.

Partitions with random representatives. This makes the equivalence-class idea from the paper a first-class feature. A value can be a named partition that draws a concrete value at run time from a recorded seed:

```

space = TestSpace(
    x = [Partition(:negative, rng -> -1e3 * rand(rng)),
         Partition(:tiny,      rng -> 1e-12 * rand(rng)),
         0.0, Inf],
    alg = [:rk4, :midpoint],
)

```

Coverage is guaranteed over partitions, while values vary from run to run. This is the partnership with property-based testing described in Section 2, and a package extension could accept `Supposition.jl` generators directly.

Budgets and rolling coverage. `all_pairs(space; max_cases = 12)` would return the best 12 cases in coverage order and report the share covered. `GND(seed = run_number)` in CI would give a different valid pairwise matrix on each run. The summary could then report how much three-way coverage has built up

across recorded runs.

8 Making the solvers reliable

8.1 Two ways generation fails

Constrained generation fails in two ways, and one design fixes both.

1. **Implicit constraints.** A tuple passes the rule on its own, but no complete, allowed test contains it. The engines treat it as required. IPOG then crashes, and GND waits forever. This accounts for 28 of 43 pairwise crashes and 65 of 69 three-way crashes.
2. **Greedy dead ends.** Every required tuple is feasible, but values are placed after checking only that the row is not *yet* forbidden. A row can become impossible to complete. This accounts for 15 and 4 crashes.

8.2 The fix: feasibility first, then keep every row completable

Four parts.

1. **Feasibility pass.** A tuple is required only if some complete, allowed test contains it. Check each tuple once, before generation. Report the rest by name, and distinguish “forbidden by rule k ” from “impossible because rules combine”.
2. **Completeness invariant.** Whenever an engine sets a value, require that the row can still be completed. In IPOG that means three sites: `choose_last_parameter_filter!` (line 34), `insert_tuple_into_tests_filter`, and `fill_remaining_missing_values_filter!` (line 226). In GND it means candidate construction.
3. **Guaranteed progress.** With parts 1 and 2, IPOG cannot reach a dead end. GND always has a covering candidate, because the first uncovered tuple is itself completable. Every remaining loop should still stop, with a diagnosis, if a pass makes no progress.
4. **Final validation.** Check every complete row, including must-run cases, and report any case that breaks a rule. Reject it unless the user insists.

My prototype implements parts 1–3 around a simple greedy generator. It uses a depth-first search to check completeness. It returned valid, fully covering designs for all 1,000 random problems, including every problem that crashed IPOG, and it produced the “impossible: rules 1 and 2 together” report shown earlier. It is not tuned for design size. Its purpose is to show that the completeness invariant alone removes both failure classes.

Cost. The completeness check is a backtracking search over unassigned parameters. In the worst case it is exponential, but in practice it is cheap, because rules are local. With scoped rules, only parameters connected to the row’s assigned parameters through the rule graph need to be searched. Unconstrained parameters can be filled in freely, and results can be cached on the constrained part of the row. This mirrors the optimizations in IPOG-C, which reduced constraint-checking cost by one to two orders of magnitude and ships in NIST’s ACTS [3]. IPOG-C is the closest published model for this change.

8.3 Rules the solver can understand

Scoped rules, meaning functions whose argument names are parameter names, fix several problems at once:

- The engine calls a rule only when all of its parameters are assigned, so nothing never appears. A real nothing value is just a value.
- The scope is known, which the cheap completability check above depends on.
- Messages can name the rule (“rule 2: os, arch”), and a label can be attached by writing a rule as a pair, “no CUDA on Windows” => rule.

`Base.method_argnames` recovers the names. I checked that it works on Julia 1.12 for both anonymous and named functions. It is internal to Base, however, so the package should also accept an explicit form such as `(:os, :gpu) => (o, g) -> o == :mac && g`. The current whole-row disallow can stay for compatibility. For it, the engine would check complete rows only and search over all parameters, which is slower but correct.

8.4 Where Z3 fits

The `z3_example.jl` draft translates one kind of expression: a parameter compared with a literal, combined with `&&` and `||`. That is narrower than what users can write today, which is any Julia predicate. It also adds a large binary dependency, for domains of a handful of values where a short search is enough. I would keep Z3 out of the core. If very large constrained spaces appear, an optional package extension could translate scoped rules into SMT and speed up the same completability question.

8.5 Smaller fixes

- Change `n_way` to `n_way - 1` in the GND match condition (`src/coverage_matrix.jl:193`). Give GND a fixed default seed, record it in the design, and print it when a test fails.
- Copy the wayness dictionary rather than modifying it. Accept ranges and tuples in it. Ignore, or explain, entries at the base strength instead of asserting.
- Allow single-valued parameters, and a single parameter at strength 1. Check that the strength does not exceed the number of parameters.
- Validate must-run cases by name and value before generating.
- Have `full_factorial` estimate its size and refuse beyond a limit unless asked.
- Say when an excursion’s base case is forbidden.
- Replace column slices in hot loops with views or explicit loops, which would remove most of the multi-gigabyte allocation.
- Add the random constrained problems and the independent checker to the test suite. They would have caught both failure classes and the GND bug.

9 Documentation and first contact

- **The README’s first screen** should state the situation (“many options; too many combinations to test all”), the guarantee in one sentence, one example with named parameters and its printed summary, and a line on when to use something else.
- **Organize the manual by job, not by algorithm.** Useful recipes: test a function with many options; test generic code across types; plan a CI matrix; run a simulation campaign from files; audit an existing suite; diagnose a failure; test invalid inputs; combine with property-based testing. Algorithm pages can stay, further down.

- **Move the advice on choosing values into the manual.** Equivalence classes, boundary values, and how many values to list are the most important human decisions in combinatorial testing, and today they are explained well only in the paper.
- **Correct the engine page.** Make every example a doctest; that alone would have caught the broken wayness example.
- **Add the one-page agent guide** described in Section 6.

10 Roadmap

Table 10: A phased plan. Each phase is useful by itself.

Phase	Work	Scale
0. Correct	GND one-token fix and default seed; feasibility pass and completability checks; no loops without progress; must-run case validation; wayness fixes; user-vocabulary errors; engine documentation; random-rule regression tests.	1–2 weeks
1. Speak up	Design object with a printed summary; named parameters and NamedTuple cases; scoped rules; coverage; must_include and strength names, with aliases.	2–4 weeks
2. Close the loop	run_cases and file-based outcomes; diagnose and followups; extend and topup; max_cases and coverage ordering.	3–4 weeks
3. Reach	Tables.jl extension; CSV/TOML; GitHub matrix; as_code; Invalid; Partition; rolling coverage; agent guide; job-oriented recipes.	Ongoing

Phase 0 is the one I would not postpone. A user with rules is exactly the user who needs this package most, and in my random constrained problems about one in nine ended in a confusing crash; GND users get a process that never ends.

11 Closing

The hard part of this package, the covering algorithms, is already done and largely correct. It is fast, it has few dependencies, and its documentation and paper explain the ideas generously. What it lacks is a voice while it runs. It never tells users what it promised, what it left out, what happened, or what they could do next.

The users this package could serve best are people who already know that some combinations matter and cannot afford all of them: scientists with expensive runs, maintainers of generic code, engineers paying for CI, and agents writing tests for them. Each of them would trust the tool more if it showed its work, and would get more from it if it helped with the step after generation. Named parameters, a summary that states the guarantee, reliable rules, and a diagnosis after failure would make UnitTestDesign.jl the tool people reach for at the moment the loop explodes.

A Reproducing the experiments

All scripts ran against the working tree (v0.4.0, commit e0504d6), with the package added by `Pkg.develop` to a scratch environment. The experiments changed no repository files, and the scripts live outside the repository; the constraint and GND scripts are the ones worth turning into regression tests. The checker in `verify.jl` enumerates the full factorial to find feasible tuples, so it is limited to small problems by design.

Script	What it measures
e1.jl	First contact: REPL output, exported names, edge-case errors
e2.jl, e3.jl	1,000 random constrained problems; classification of crashes
e4.jl	GND on the implicit-constraint example under a 45-second watchdog
e5.jl	wayness example, dictionary mutation, GND reproducibility, IPOG stability
e6.jl	Mixed-strength verification; triples covered by pairwise designs; coverage by prefix
e7.jl, e8.jl	Time, allocation, and size for IPOG and GND; GND sensitivity to M
e9.jl	GND sizes before and after the one-token patch (patched in-session only)
e10.jl, e16.jl	Test.jl output for a failing design; localization and masked pairs
e11.jl	Excursion base case, rule-violating must-run cases, value types
e12.jl, proto.jl	Constraint-safe prototype on the same 1,000 problems
e14.jl, e17.jl	Figures for the CI example and the strength comparison
e15.jl	Silent coverage loss when nothing is a real value

B Findings by location

Location	Finding
src/coverage_matrix.jl:193	GND scoring uses <code>n_way</code> ; needs <code>n_way - 1</code> .
src/greedy_tuples.jl:195, 201	Loops with no progress check; hang on uncoverable tuples or all-forbidden candidates.
src/greedy_tuples.jl:250, 256	The same loops in the mixed-strength variant.
src/parameter_order.jl:128, 170	Unplaceable tuples appended as rows without a completability check.
src/parameter_order.jl:224–226	Final fill leaves zeros when no value is allowed; leads to <code>p[0]</code> .
src/parameter_order.jl:436, src/excursions.jl:32	The user's wayness dictionary is modified.
src/coverage_matrix.jl:426	Assertion instead of a message when an extra strength equals the base.
src/factorial_interface.jl:61	GND's default RNG is unseeded and unrecorded.
src/factorial_interface.jl:85	nothing marks unassigned parameters; ambiguous with real nothing.
src/factorial_interface.jl:100	Must-run values looked up with <code>indexin</code> ; a miss surfaces as a convert error.
src/factorial_interface.jl:219	Single-valued parameters rejected.
src/full_factorial.jl:5	Allocates the full result before any size check.
docs/src/man/guide.md	wayness example throws.
docs/src/man/engines.md	Describes GND as shorter; measured longer.

References

- [1] D. R. Kuhn, D. R. Wallace, and A. M. Gallo Jr. Software fault interactions and implications for software testing. *IEEE Transactions on Software Engineering*, 30(6):418–421, 2004.
- [2] Y. Lei, R. Kacker, D. R. Kuhn, V. Okun, and J. Lawrence. IPOG/IPOG-D: Efficient test generation for multi-way combinatorial testing. *Software Testing, Verification and Reliability*, 18(3):125–148, 2008.
- [3] L. Yu, Y. Lei, M. Nourozborazjany, R. N. Kacker, and D. R. Kuhn. An efficient algorithm for constraint handling in combinatorial test generation. In *IEEE Sixth International Conference on Software Testing, Verification and Validation (ICST)*, 2013.

- [4] D. R. Kuhn, I. Dominguez Mendoza, R. N. Kacker, and Y. Lei. Combinatorial coverage measurement concepts and applications. In *Second International Workshop on Combinatorial Testing (IWCT)*, 2013. Tool: <https://github.com/usnistgov/combinatorial-testing-tools>.
- [5] L. S. Ghandehari et al. A combinatorial testing-based approach to fault localization. *IEEE Transactions on Software Engineering*, 46(6):616–645, 2020.
- [6] C. Yilmaz, E. Dumlu, M. B. Cohen, and A. Porter. Reducing masking effects in combinatorial interaction testing: A feedback driven adaptive approach. *IEEE Transactions on Software Engineering*, 40(1):43–66, 2014.
- [7] Microsoft. PICT: Pairwise Independent Combinatorial Testing, user guide. <https://github.com/microsoft/pict/blob/main/doc/pict.md>.
- [8] Seelengrab (GitHub author). Supposition.jl: choice-sequence based property-based testing for Julia. <https://github.com/Seelengrab/Supposition.jl>.
- [9] GitHub Docs. Running variations of jobs in a workflow (a matrix generates at most 256 jobs per workflow run). <https://docs.github.com/en/actions/using-jobs/using-a-matrix-for-your-jobs>.